# Filtros no ASP.NET Core

Artigo • 03/11/2022 • 49 minutos para o fim da leitura

Por Kirk Larkin , Rick Anderson , Tom Dykstra e Steve Smith

*Os filtros* no ASP.NET Core permitem que o código seja executado antes ou depois de estágios específicos no pipeline de processamento de solicitação.

O filtros internos lidam com tarefas como:

- Autorização, impedindo o acesso a recursos para os quais um usuário não está autorizado.
- Cache de resposta, curto-circuito do pipeline de solicitação para retornar uma resposta armazenada em cache.

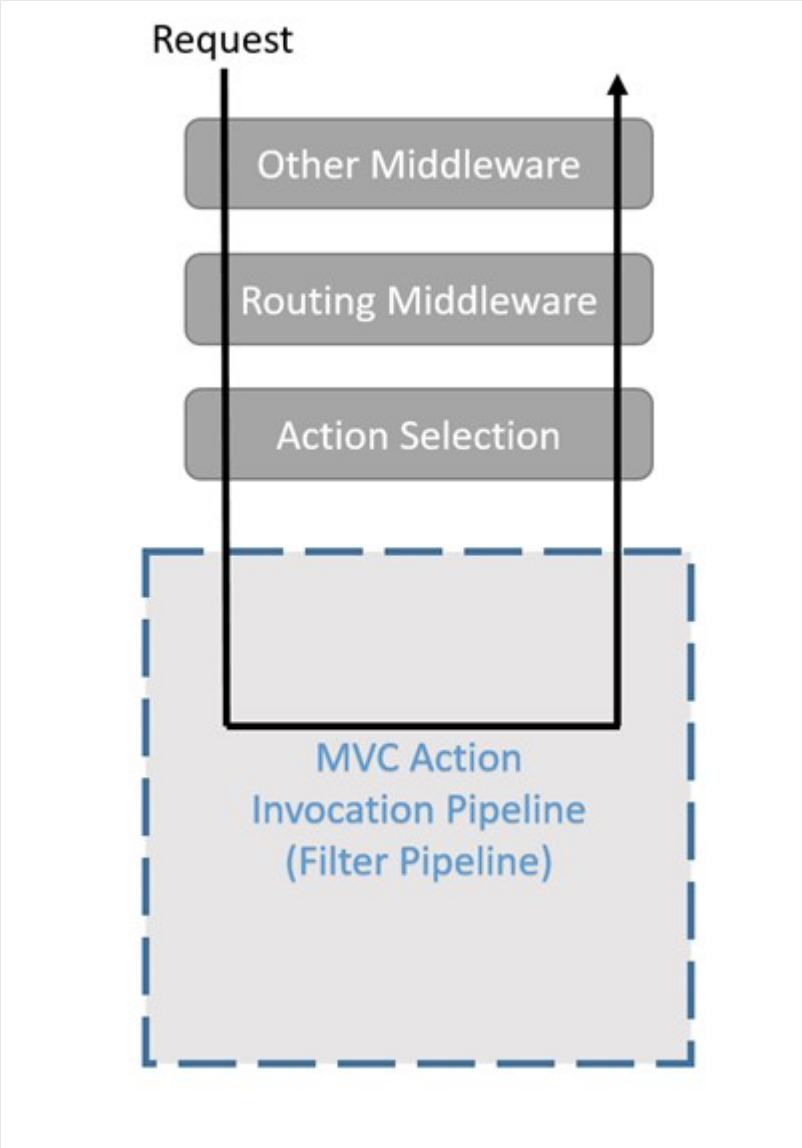
É possível criar filtros personalizados para lidar com interesses paralelos. Entre os exemplos de interesses paralelos estão o tratamento de erros, cache, configuração, autorização e registro em log. Filtros evitam a duplicação do código. Por exemplo, um filtro de exceção de tratamento de erro poderia consolidar o tratamento de erro.

Este documento se aplica a Razor Páginas, controladores de API e controladores com exibições. Os filtros não funcionam diretamente com Razor componentes. Um filtro só pode afetar indiretamente um componente quando:

- O componente é inserido em uma página ou exibição.
- A página ou o controlador e a exibição usam o filtro.

## Como os filtros funcionam

Os filtros são executados dentro do *pipeline de invocação de ações do ASP.NET Core*, às vezes chamado de *pipeline de filtros*. O pipeline de filtro é executado após ASP.NET Core seleciona a ação a ser executada:



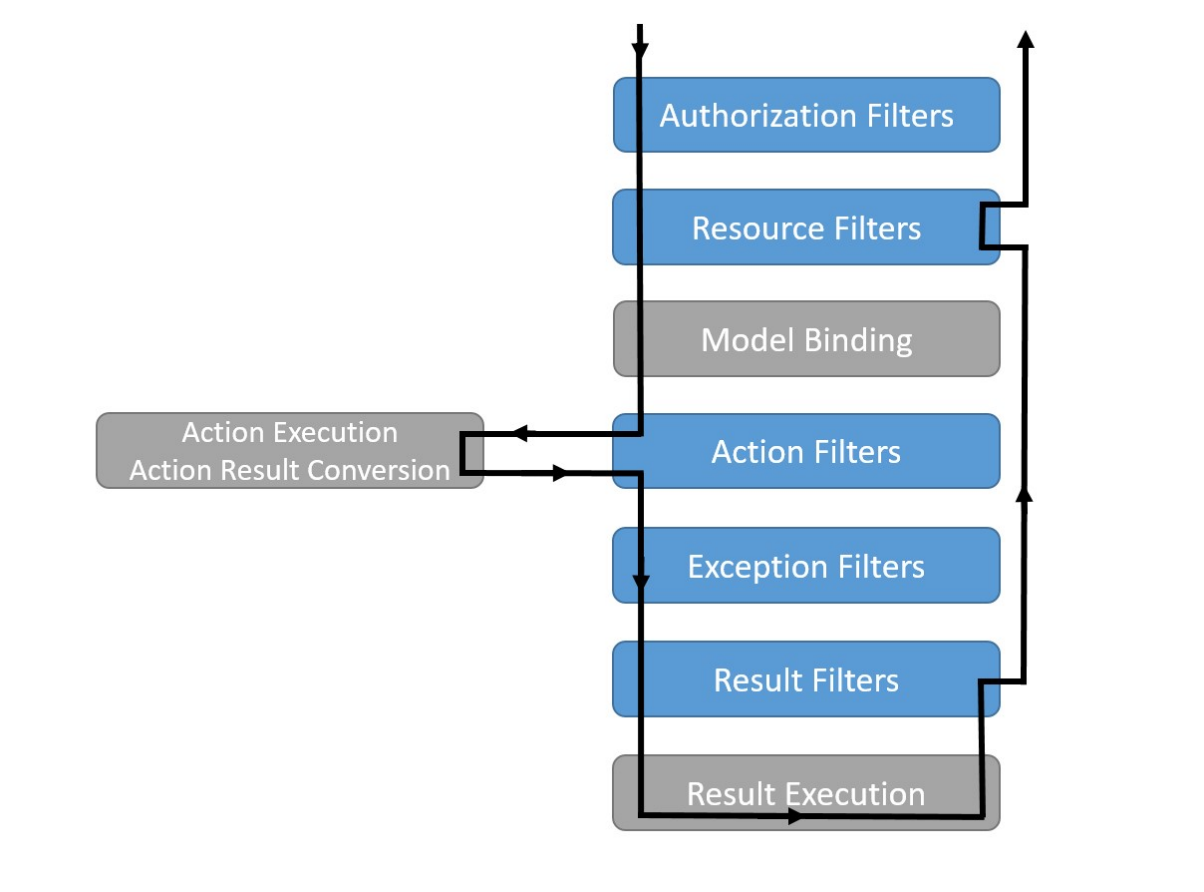

## Tipos de filtro

Cada tipo de filtro é executado em um estágio diferente no pipeline de filtros:

- Filtros de autorização:
  - Execute primeiro.
  - Determine se o usuário está autorizado para a solicitação.
  - Curto-circuito do pipeline se a solicitação não estiver autorizada.
- Filtros de recurso:
  - Execute após a autorização.
  - OnResourceExecuting executa o código antes do restante do pipeline de filtro. Por exemplo, `OnResourceExecuting` executa o código antes da associação de modelo.
  - OnResourceExecuted executa o código após a conclusão do restante do pipeline.

- Filtros de ação:
  - Execute imediatamente antes e depois que um método de ação for chamado.
  - Pode alterar os argumentos passados para uma ação.
  - Pode alterar o resultado retornado da ação.
  - **Não** há suporte no Razor Pages.
- Os filtros de exceção aplicam políticas globais a exceções sem tratamento que ocorrem antes que o corpo da resposta seja gravado.
- Filtros de resultado:
  - Execute imediatamente antes e depois da execução dos resultados da ação.
  - Execute somente quando o método de ação for executado com êxito.
  - São úteis para a lógica que deve envolver a exibição ou a execução do formatador.

O diagrama a seguir mostra como os tipos de filtro interagem no pipeline de filtro:




Razor As páginas também dão suporte Razor a filtros de página, que são executados antes e depois de um Razor manipulador de página.

# Implementação

Os filtros dão suporte a implementações síncronas e assíncronas por meio de diferentes definições de interface.

Os filtros síncronos são executados antes e depois do estágio do pipeline. Por exemplo, OnActionExecuting é chamado antes que o método de ação seja chamado. OnActionExecuted é chamado depois que o método de ação retorna:

C#

```
public class SampleActionFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        // Do something before the action executes.
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
        // Do something after the action executes.
    }
}
```

Os filtros assíncronos definem um `On-Stage-ExecutionAsync` método . Por exemplo OnActionExecutionAsync:

C#

```
public class SampleAsyncActionFilter : IAsyncActionFilter
{
    public async Task OnActionExecutionAsync(
        ActionExecutingContext context, ActionExecutionDelegate next)
    {
        // Do something before the action executes.
        await next();
        // Do something after the action executes.
    }
}
```

No código anterior, o `SampleAsyncActionFilter` tem um ActionExecutionDelegate, `next`, que executa o método de ação.

## Vários estágios do filtro

É possível implementar interfaces para vários estágios do filtro em uma única classe. Por exemplo, a ActionFilterAttribute classe implementa:

- Síncrono: IActionFilter e IResultFilter
- Assíncrono: IAsyncActionFilter e IAsyncResultFilter
- IOrderedFilter

Implemente **ou** a versão assíncrona ou a versão síncrona de uma interface de filtro, **não** ambas. Primeiro, o runtime verifica se o filtro implementa a interface assíncrona e, se for esse o caso, a chama. Caso contrário, ela chama os métodos da interface síncrona. Se as interfaces síncrona e assíncrona forem implementadas em uma classe, somente o método assíncrono será chamado. Ao usar classes abstratas como ActionFilterAttribute, substitua apenas os métodos síncronos ou os métodos assíncronos para cada tipo de filtro.

## Atributos de filtro internos

O ASP.NET Core inclui filtros internos baseados em atributos que podem ser organizados em subclasses e personalizados. Por exemplo, o filtro de resultado a seguir adiciona um cabeçalho à resposta:

C#

```
public class ResponseHeaderAttribute : ActionFilterAttribute
{
    private readonly string _name;
    private readonly string _value;

    public ResponseHeaderAttribute(string name, string value) =>
        (_name, _value) = (name, value);

    public override void OnResultExecuting(ResultExecutingContext con-
text)
    {
        context.HttpContext.Response.Headers.Add(_name, _value);

        base.OnResultExecuting(context);
    }
}
```

Os atributos permitem que os filtros aceitem argumentos, conforme mostrado no exemplo acima. Aplique o `ResponseHeaderAttribute` a um controlador ou método de ação e especifique o nome e o valor do cabeçalho HTTP:

C#

```
[ResponseHeader("Filter-Header", "Filter Value")]
public class ResponseHeaderController : ControllerBase
```

```
{
    public IActionResult Index() =>
        Content("Examine the response headers using the F12 developer to-
ols.");

    // ...
```

Use uma ferramenta como as ferramentas de desenvolvedor do navegador para examinar os cabeçalhos. Em **Cabeçalhos de Resposta**, `filter-header: Filter Value` é exibido.

O código a seguir se aplica `ResponseHeaderAttribute` a um controlador e a uma ação:

C#

```
[ResponseHeader("Filter-Header", "Filter Value")]
public class ResponseHeaderController : ControllerBase
{
    public IActionResult Index() =>
        Content("Examine the response headers using the F12 developer to-
ols.");

    // ...

    [ResponseHeader("Another-Filter-Header", "Another Filter Value")]
    public IActionResult Multiple() =>
        Content("Examine the response headers using the F12 developer to-
ols.");
}
```

As respostas da ação `Multiple` incluem os seguintes cabeçalhos:

- `filter-header: Filter Value`
- `another-filter-header: Another Filter Value`

Várias interfaces de filtro têm atributos correspondentes que podem ser usados como classes base para implementações personalizadas.

Atributos de filtro:

- ActionFilterAttribute
- ExceptionFilterAttribute
- ResultFilterAttribute
- FormatFilterAttribute
- ServiceFilterAttribute
- TypeFilterAttribute

Os filtros não podem ser aplicados aos Razor métodos de manipulador de página. Eles podem ser aplicados ao Razor modelo de página ou globalmente.

# Escopos e ordem de execução dos filtros

Um filtro pode ser adicionado ao pipeline com um de três *escopos*:

- Usando um atributo em um controlador ou Razor Página.
- Usando um atributo em uma ação do controlador. Os atributos de filtro não podem ser aplicados aos Razor métodos do manipulador Pages.
- Globalmente para todos os controladores, ações e Razor Páginas, conforme mostrado no código a seguir:

C#

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews(options =>
{
    options.Filters.Add<GlobalSampleActionFilter>();
});
```

## Ordem padrão de execução

Quando há vários filtros para um determinado estágio do pipeline, o escopo determina a ordem padrão de execução dos filtros. Filtros globais circundam filtros de classe, que, por sua vez, circundam filtros de método.

Como resultado do aninhamento de filtro, o código *posterior* dos filtros é executado na ordem inversa do código *anterior*. A sequência de filtro:

- O código *anterior* dos filtros globais.
  - O código *anterior* dos filtros de controlador.
    - O código *anterior* dos filtros de método de ação.
    - O código *posterior* dos filtros de método de ação.
  - O código *posterior* dos filtros de controlador.
- O código *posterior* dos filtros globais.

O exemplo a seguir ilustra a ordem na qual os métodos de filtro são executados para filtros de ação síncrona:

| Sequência | Escopo do filtro | Método do filtro |
|---|---|---|
| 1 | Global | `OnActionExecuting` |
| 2 | Controller | `OnActionExecuting` |
| 3 | Ação | `OnActionExecuting` |
| 4 | Ação | `OnActionExecuted` |
| 5 | Controller | `OnActionExecuted` |
| 6 | Global | `OnActionExecuted` |

## Filtros no nível do controlador

Cada controlador que herda de Controller inclui os OnActionExecutingmétodos , OnActionExecutionAsynce OnActionExecuted . Esses métodos encapsulam os filtros executados para uma determinada ação:

- `OnActionExecuting` é executado antes de qualquer um dos filtros da ação.
- `OnActionExecuted` é executado após todos os filtros da ação.
- `OnActionExecutionAsync` é executado antes de qualquer um dos filtros da ação. Código após uma chamada a ser `next` executada após os filtros da ação.

A seguinte `ControllerFiltersController` classe:

- Aplica o `SampleActionFilterAttribute` (`[SampleActionFilter]`) ao controlador.
- Substitui `OnActionExecuting` e `OnActionExecuted`.

C#

```
[SampleActionFilter]
public class ControllerFiltersController : Controller
{
    public override void OnActionExecuting(ActionExecutingContext con-
text)
    {
        Console.WriteLine(
            $"- {nameof(ControllerFiltersController)}.{name-
of(OnActionExecuting)}");

        base.OnActionExecuting(context);
    }

    public override void OnActionExecuted(ActionExecutedContext context)
    {
```

```
        Console.WriteLine(
            $"- {nameof(ControllerFiltersController)}.{name-
of(OnActionExecuted)}");

        base.OnActionExecuted(context);
    }

    public IActionResult Index()
    {
        Console.WriteLine(
            $"- {nameof(ControllerFiltersController)}.{nameof(Index)}");

        return Content("Check the Console.");
    }
}
```

Navegar até `https://localhost:<port>/ControllerFilters` executa o seguinte código:

- `ControllerFiltersController.OnActionExecuting`
  - `GlobalSampleActionFilter.OnActionExecuting`
    - `SampleActionFilterAttribute.OnActionExecuting`
      - `ControllerFiltersController.Index`
    - `SampleActionFilterAttribute.OnActionExecuted`
  - `GlobalSampleActionFilter.OnActionExecuted`
- `ControllerFiltersController.OnActionExecuted`

Os filtros no nível do controlador definem a propriedade Order como `int.MinValue`. Os filtros no nível do controlador **não** podem ser definidos para serem executados após os filtros aplicados aos métodos. A ordem é explicada na próxima seção.

Para Razor Páginas, consulte Implementar Razor filtros de página substituindo métodos de filtro.

## Substituir a ordem padrão

É possível substituir a sequência padrão de execução implementando IOrderedFilter. `IOrderedFilter` expõe a propriedade Order que tem precedência sobre o escopo para determinar a ordem da execução. Um filtro com um valor de `Order` menor:

- Executa o código *anterior* antes de um filtro com um valor mais alto de `Order`.
- Executa o código *posterior* após um filtro com um valor mais alto de `Order`.

No exemplo de filtros de nível do controlador, tem escopo global, portanto, `GlobalSampleActionFilter` ele é executado antes `SampleActionFilterAttribute` de , que tem o escopo do controlador. Para executar `SampleActionFilterAttribute` primeiro, defina sua ordem como `int.MinValue`:

C#

```
[SampleActionFilter(Order = int.MinValue)]
public class ControllerFiltersController : Controller
{
    // ...
}
```

Para que o filtro global seja `GlobalSampleActionFilter` executado primeiro, defina como `Order` `int.MinValue`:

C#

```
builder.Services.AddControllersWithViews(options =>
{
    options.Filters.Add<GlobalSampleActionFilter>(int.MinValue);
});
```

# Cancelamento e curto-circuito

O pipeline de filtro pode sofrer um curto-circuito por meio da configuração da propriedade Result no parâmetro ResourceExecutingContext fornecido ao método do filtro. Por exemplo, o filtro de recurso a seguir impede que o restante do pipeline seja executado:

C#

```
public class ShortCircuitingResourceFilterAttribute : Attribute,
IResourceFilter
{
    public void OnResourceExecuting(ResourceExecutingContext context)
    {
        context.Result = new ContentResult
        {
            Content = nameof(ShortCircuitingResourceFilterAttribute)
        };
    }

    public void OnResourceExecuted(ResourceExecutedContext context) { }
}
```

No código a seguir, os filtros `[ShortCircuitingResourceFilter]` e `[ResponseHeader]` têm como destino o método de ação `Index`. O `ShortCircuitingResourceFilterAttribute` filtro:

- É executado primeiro, porque ele é um filtro de recurso e `ResponseHeaderAttribute` é um filtro de ação.
- Causa um curto-circuito no restante do pipeline.

Portanto, o filtro `ResponseHeaderAttribute` nunca é executado para a ação `Index`. Esse comportamento seria o mesmo se os dois filtros fossem aplicados no nível do método de ação, desde que `ShortCircuitingResourceFilterAttribute` fosse executado primeiro. O `ShortCircuitingResourceFilterAttribute` é executado primeiro devido ao seu tipo de filtro:

C#

```
[ResponseHeader("Filter-Header", "Filter Value")]
public class ShortCircuitingController : Controller
{
    [ShortCircuitingResourceFilter]
    public IActionResult Index() =>
        Content($"- {nameof(ShortCircuitingController)}.{name-
of(Index)}");
}
```

# Injeção de dependência

Filtros podem ser adicionados por tipo ou por instância. Se você adicionar uma instância, ela será usada para cada solicitação. Se um tipo for adicionado, ele será ativado pelo tipo. Um filtro ativado por tipo significa:

- Uma instância é criada para cada solicitação.
- Qualquer dependência de construtor é preenchida pela injeção de dependência (DI).

Filtros que são implementados como atributos e adicionados diretamente a classes de controlador ou métodos de ação não podem ter dependências de construtor fornecidas pela DI (injeção de dependência). As dependências do construtor não podem ser fornecidas pela DI porque os atributos devem ter seus parâmetros de construtor fornecidos onde são aplicados.

Os filtros a seguir são compatíveis com dependências de construtor fornecidas pela

injeção de dependência:

- ServiceFilterAttribute
- TypeFilterAttribute
- IFilterFactory implementado no atributo.

Os filtros anteriores podem ser aplicados a um controlador ou a uma ação.

Os agentes estão disponíveis na DI. No entanto, evite criar e usar filtros apenas para fins de registro em log. O registro em log da estrutura interna normalmente fornece o que é necessário para o registro em log. Registro em log adicionado aos filtros:

- Deve se concentrar em questões de domínio de negócios ou comportamento específico ao filtro.
- **Não** deve registrar ações ou outros eventos da estrutura. Os filtros internos já registram ações e eventos de estrutura.

## ServiceFilterAttribute

Os tipos de implementação do filtro de serviço são registrados em `Program.cs`. Um ServiceFilterAttribute recupera uma instância do filtro da DI.

O código a seguir mostra a `LoggingResponseHeaderFilterService` classe , que usa DI:

C#

```
public class LoggingResponseHeaderFilterService : IResultFilter
{
    private readonly ILogger _logger;

    public LoggingResponseHeaderFilterService(
            ILogger<LoggingResponseHeaderFilterService> logger) =>
        _logger = logger;

    public void OnResultExecuting(ResultExecutingContext context)
    {
        _logger.LogInformation(
            $"- {nameof(LoggingResponseHeaderFilterService)}.{name-
of(OnResultExecuting)}");

        context.HttpContext.Response.Headers.Add(
            nameof(OnResultExecuting), name-
of(LoggingResponseHeaderFilterService));
    }

    public void OnResultExecuted(ResultExecutedContext context)
    {
```

```
        _logger.LogInformation(
            $"- {nameof(LoggingResponseHeaderFilterService)}.{name-
of(OnResultExecuted)}");
    }
}
```

No código a seguir, `LoggingResponseHeaderFilterService` é adicionado ao contêiner de DI:

```
C#

builder.Services.AddScoped<LoggingResponseHeaderFilterService>();
```

No código a seguir, o atributo `ServiceFilter` recupera uma instância do filtro `LoggingResponseHeaderFilterService` da DI:

```
C#

[ServiceFilter(typeof(LoggingResponseHeaderFilterService))]
public IActionResult WithServiceFilter() =>
    Content($"- {nameof(FilterDependenciesController)}.{name-
of(WithServiceFilter)}");
```

Ao usar `ServiceFilterAttribute`, configuração ServiceFilterAttribute.IsReusable:

- Fornece uma dica de que a instância de filtro *pode* ser reutilizada fora do escopo de solicitação em que foi criada. O runtime do ASP.NET Core não garante:
  - Que uma única instância do filtro será criada.
  - Que o filtro não será solicitado novamente no contêiner de DI em algum momento posterior.
- Não deve ser usado com um filtro que dependa de serviços com um tempo de vida diferente de singleton.

ServiceFilterAttribute implementa IFilterFactory. `IFilterFactory` expõe o método CreateInstance para criar uma instância de IFilterMetadata. `CreateInstance` carrega o tipo especificado na DI.

## TypeFilterAttribute

O TypeFilterAttribute é semelhante ao ServiceFilterAttribute, mas seu tipo não é resolvido diretamente por meio do contêiner de DI. Ele cria uma instância do tipo usando Microsoft.Extensions.DependencyInjection.ObjectFactory.

Como os tipos `TypeFilterAttribute` não são resolvidos diretamente do contêiner de DI:

- Os tipos referenciados usando o `TypeFilterAttribute` não precisam ser registrados no contêiner de DI. Eles têm suas dependências atendidas pelo contêiner de DI.
- Opcionalmente, o `TypeFilterAttribute` pode aceitar argumentos de construtor para o tipo.

Ao usar `TypeFilterAttribute`, configuração TypeFilterAttribute.IsReusable:

- Fornece uma dica de que a instância de filtro *pode* ser reutilizada fora do escopo de solicitação em que foi criada. O runtime do ASP.NET Core não fornece garantias de que uma única instância do filtro será criada.

- Não deve ser usado com um filtro que dependa dos serviços com um tempo de vida diferente de singleton.

O exemplo a seguir mostra como passar argumentos para um tipo usando `TypeFilterAttribute`:

C#

```
[TypeFilter(typeof(LoggingResponseHeaderFilter),
    Arguments = new object[] { "Filter-Header", "Filter Value" })]
public IActionResult WithTypeFilter() =>
    Content($"- {nameof(FilterDependenciesController)}.{name-
of(WithTypeFilter)}");
```

# Filtros de autorização

Filtros de autorização:

- São os primeiros filtros executados no pipeline de filtro.
- Controlam o acesso aos métodos de ação.
- Têm um método anterior, mas não têm um método posterior.

Filtros de autorização personalizados exigem uma estrutura de autorização personalizada. Prefira configurar as políticas de autorização ou escrever uma política de autorização personalizada em vez de escrever um filtro personalizado. O filtro de autorização interno:

- Chama o sistema de autorização.

- Não autoriza solicitações.

**Não** gera exceções dentro de filtros de autorização:

- A exceção não será tratada.
- Os filtros de exceção não tratarão a exceção.

Considere a possibilidade de emitir um desafio quando ocorrer uma exceção em um filtro de autorização.

Saiba mais sobre Autorização.

# Filtros de recurso

Filtros de recurso:

- Implementam a interface IResourceFilter ou IAsyncResourceFilter.
- A execução encapsula grande parte do pipeline de filtro.
- Somente os Filtros de autorização são executados antes dos filtros de recurso.

Os filtros de recursos são úteis para causar um curto-circuito na maior parte do pipeline. Por exemplo, um filtro de cache pode evitar o restante do pipeline em uma ocorrência no cache.

Exemplos de filtros de recurso:

- O filtro de recurso em curto-circuito mostrado anteriormente.
- DisableFormValueModelBindingAttribute :
  - Impede o model binding de acessar os dados do formulário.
  - Usado para uploads de arquivos grandes para impedir que os dados de formulário sejam lidos na memória.

# Filtros de ação

Os filtros de ação **não** se aplicam a Razor Páginas. Razor O Pages dá IPageFilter suporte a e IAsyncPageFilter. Para obter mais informações, consulte Métodos de filtro para Razor Pages.

Filtros de ação:

- Implementam a interface IActionFilter ou IAsyncActionFilter.
- A execução deles envolve a execução de métodos de ação.

O código a seguir mostra um exemplo de filtro de ação:

```C#
public class SampleActionFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        // Do something before the action executes.
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
        // Do something after the action executes.
    }
}
```

A classe ActionExecutingContext fornece as seguintes propriedades:

- ActionArguments – permite ler as entradas para um método de ação.
- Controller – permite a manipulação da instância do controlador.
- Result – configuração de `Result` execução de curtos-circuitos do método de ação e dos filtros de ação posteriores.

Gerar uma exceção em um método de ação:

- Impede a execução de filtros subsequentes.
- Ao contrário da configuração `Result`, é tratada como uma falha e não como um resultado bem-sucedido.

A classe ActionExecutedContext fornece `Controller` e `Result`, além das seguintes propriedades:

- Canceled – verdadeiro se a execução da ação tiver sofrido curto-circuito por outro filtro.
- Exception – não será nulo se a ação ou um filtro de ação executado antes tiver apresentado uma exceção. Definir essa propriedade como nulo:
  - Trata efetivamente a exceção.
  - `Result` é executado como se tivesse retornado do método de ação.

Para um `IAsyncActionFilter`, uma chamada para o ActionExecutionDelegate:

- Executa todos os próximos filtros de ação e o método de ação.
- Retorna `ActionExecutedContext`.

Para fazer um curto-circuito, atribua Microsoft.AspNetCore.Mvc.Filters.ActionExecutingContext.Result a uma instância de resultado e não chame o `next` (`ActionExecutionDelegate`).

A estrutura fornece um ActionFilterAttribute abstrato que você pode colocar em uma subclasse.

O filtro de ação `OnActionExecuting` pode ser usado para:

- Validar o estado do modelo.
- Retornar um erro se o estado for inválido.

C#

```
public class ValidateModelAttribute : ActionFilterAttribute
{
    public override void OnActionExecuting(ActionExecutingContext con-
text)
    {
        if (!context.ModelState.IsValid)
        {
            context.Result = new BadRequestObjectResult(con-
text.ModelState);
        }
    }
}
```

**Observação**

Os controladores anotados com o atributo validam automaticamente o `[ApiController]` estado do modelo e retornam uma resposta 400. Para obter mais informações, veja **Respostas automáticas HTTP 400**.

O método `OnActionExecuted` é executado após o método de ação:

- E pode ver e manipular os resultados da ação por meio da propriedade Result.
- Canceled é definido como verdadeiro se a execução da ação tiver sofrido curto-circuito por outro filtro.
- Exception é definido como um valor não nulo se a ação ou um filtro de ação posterior tiver apresentado uma exceção. Definindo `Exception` como nulo:
  - Trata efetivamente uma exceção.
  - `ActionExecutedContext.Result` é executado como se fosse retornado normalmente do método de ação.

# Filtros de exceção

Filtros de exceção:

- Implementam IExceptionFilter ou IAsyncExceptionFilter.
- Podem ser usados para implementar políticas de tratamento de erro comuns.

O filtro de exceção de exemplo a seguir exibe detalhes sobre exceções que ocorrem quando o aplicativo está em desenvolvimento:

C#

```
public class SampleExceptionFilter : IExceptionFilter
{
    private readonly IHostEnvironment _hostEnvironment;

    public SampleExceptionFilter(IHostEnvironment hostEnvironment) =>
        _hostEnvironment = hostEnvironment;

    public void OnException(ExceptionContext context)
    {
        if (!_hostEnvironment.IsDevelopment())
        {
            // Don't display exception details unless running in
Development.
            return;
        }

        context.Result = new ContentResult
        {
            Content = context.Exception.ToString()
        };
    }
}
```

O código a seguir testa o filtro de exceção:

C#

```
[TypeFilter(typeof(SampleExceptionFilter))]
public class ExceptionController : Controller
{
    public IActionResult Index() =>
        Content($"- {nameof(ExceptionController)}.{nameof(Index)}");
}
```

Filtros de exceção:

- Não têm eventos anteriores nem posteriores.
- Implementam OnException ou OnExceptionAsync.
- Manipule exceções sem tratamento que ocorrem na Razor criação de página ou controlador, model binding, filtros de ação ou métodos de ação.
- **Não** capturam as exceções que ocorrem em filtros de recurso, em filtros de resultado ou na execução do resultado de MVC.

Para manipular uma exceção, defina a ExceptionHandled propriedade `true` como ou atribua a Result propriedade . Isso interrompe a propagação da exceção. Um filtro de exceção não pode transformar uma exceção em "êxito". Somente um filtro de ação pode fazer isso.

Filtros de exceção:

- São bons para interceptar as exceções que ocorrem nas ações.
- Não são tão flexíveis quanto o middleware de tratamento de erro.

Prefira o middleware para tratamento de exceção. Use filtros de exceção apenas quando o tratamento de erros *for diferente* com base no método de ação chamado. Por exemplo, um aplicativo pode ter métodos de ação para os pontos de extremidade da API e para modos de exibição/HTML. Os pontos de extremidade da API podem retornar informações de erro como JSON, enquanto as ações baseadas em exibição podem retornar uma página de erro como HTML.

# Filtros de resultado

Filtros de resultado:

- Implemente uma interface:
  - IResultFilter ou IAsyncResultFilter
  - IAlwaysRunResultFilter ou IAsyncAlwaysRunResultFilter
- A execução deles envolve a execução de resultados de ação.

## IResultFilter e IAsyncResultFilter

O código a seguir mostra um filtro de resultado de exemplo:

C#

```
public class SampleResultFilter : IResultFilter
{
    public void OnResultExecuting(ResultExecutingContext context)
```

```
    {
        // Do something before the result executes.
    }

    public void OnResultExecuted(ResultExecutedContext context)
    {
        // Do something after the result executes.
    }
}
```

O tipo de resultado que está sendo executado depende da ação. Uma ação que retorna uma exibição inclui todo o processamento razor como parte do ViewResult que está sendo executado. Um método de API pode executar alguma serialização como parte da execução do resultado. Saiba mais sobre os resultados da ação.

Os filtros de resultado só são executados quando uma ação ou filtro de ação produz um resultado de ação. Os filtros de resultado não são executados quando:

- Um filtro de autorização ou filtro de recurso causa um curto-circuito no pipeline.
- Um filtro de exceção trata uma exceção produzindo um resultado de ação.

O método Microsoft.AspNetCore.Mvc.Filters.IResultFilter.OnResultExecuting pode fazer o curto-circuito da execução do resultado da ação e dos filtros de resultados posteriores definindo Microsoft.AspNetCore.Mvc.Filters.ResultExecutingContext.Cancel como `true`. Grave no objeto de resposta ao fazer um curto-circuito para evitar gerar uma resposta vazia. Gerando uma exceção em `IResultFilter.OnResultExecuting`:

- Impede a execução do resultado da ação e dos filtros subsequentes.
- É tratado como uma falha em vez de um resultado bem-sucedido.

Quando o Microsoft.AspNetCore.Mvc.Filters.IResultFilter.OnResultExecuted método é executado, a resposta provavelmente já foi enviada ao cliente. Se a resposta já tiver sido enviada ao cliente, ela não poderá ser alterada.

`ResultExecutedContext.Canceled` será definido como `true` se a execução do resultado da ação tiver sofrido curto-circuito por outro filtro.

`ResultExecutedContext.Exception` será definido como um valor não nulo se o resultado da ação ou um filtro de resultado posterior tiver gerado uma exceção. A configuração `Exception` como nulo manipula efetivamente uma exceção e impede que a exceção seja lançada novamente mais tarde no pipeline. Não há nenhuma maneira confiável para gravar dados em uma resposta ao manipular uma exceção em

um filtro de resultado. Se os cabeçalhos tiverem sido liberados para o cliente quando o resultado de uma ação gerar uma exceção, não haverá mecanismo confiável para enviar um código de falha.

Para um IAsyncResultFilter, uma chamada para `await next` no ResultExecutionDelegate executa qualquer filtro de resultado posterior e o resultado da ação. Para curto-circuito, defina ResultExecutingContext.Cancel como `true` e não chame `:ResultExecutionDelegate`

C#

```
public class SampleAsyncResultFilter : IAsyncResultFilter
{
    public async Task OnResultExecutionAsync(
        ResultExecutingContext context, ResultExecutionDelegate next)
    {
        if (context.Result is not EmptyResult)
        {
            await next();
        }
        else
        {
            context.Cancel = true;
        }
    }
}
```

A estrutura fornece um `ResultFilterAttribute` abstrato que você pode colocar em uma subclasse. A classe ResponseHeaderAttribute mostrada anteriormente é um exemplo de um atributo de filtro de resultado.

## IAlwaysRunResultFilter e IAsyncAlwaysRunResultFilter

As interfaces IAlwaysRunResultFilter e IAsyncAlwaysRunResultFilter declaram uma implementação IResultFilter que é executada para todos os resultados da ação. Isso inclui os resultados da ação produzidos por:

- Filtros de autorização e filtros de recursos que fazem o curto-circuito.
- Filtros de exceção.

Por exemplo, o filtro a seguir sempre é executado e define o resultado de uma ação (ObjectResult) com um código de status *422 Entidade Não Processável* quando ocorre falha na negociação de conteúdo:

C#

```
public class UnprocessableResultFilter : IAlwaysRunResultFilter
{
    public void OnResultExecuting(ResultExecutingContext context)
    {
        if (context.Result is StatusCodeResult statusCodeResult
            && statusCodeResult.StatusCode ==
StatusCodes.Status415UnsupportedMediaType)
        {
            context.Result = new ObjectResult("Unprocessable")
            {
                StatusCode = StatusCodes.Status422UnprocessableEntity
            };
        }
    }

    public void OnResultExecuted(ResultExecutedContext context) { }
}
```

# IFilterFactory

IFilterFactory implementa IFilterMetadata. Portanto, uma instância `IFilterFactory` pode ser usada como uma instância `IFilterMetadata` em qualquer parte do pipeline de filtro. Quando o runtime se prepara para invocar o filtro, tenta convertê-lo em um `IFilterFactory`. Se essa conversão for bem-sucedida, o método CreateInstance será chamado para criar a instância `IFilterMetadata` invocada. Isso fornece um design flexível, porque o pipeline de filtro preciso não precisa ser definido explicitamente quando o aplicativo é iniciado.

`IFilterFactory.IsReusable`:

- É uma dica da fábrica de que a instância de filtro criada pela fábrica pode ser reutilizado fora do escopo da solicitação em que foi criada.
- ***Não*** deve ser usado com um filtro que dependa de serviços com um tempo de vida diferente de singleton.

O runtime do ASP.NET Core não garante:

- Que uma única instância do filtro será criada.
- Que o filtro não será solicitado novamente no contêiner de DI em algum momento posterior.

> ⚠ **Aviso**

Configure **IFilterFactory.IsReusable** apenas para retornar `true` se a origem dos filtros for inequívoca, os filtros forem sem estado e os filtros forem seguros para usar em várias solicitações HTTP. Por exemplo, não retorne filtros de DI registrados como com escopo ou transitórios se `IFilterFactory.IsReusable` retornarem `true`.

Implemente `IFilterFactory` usando implementações personalizadas de atributo como outra abordagem à criação de filtros:

C#

```
public class ResponseHeaderFilterFactory : Attribute, IFilterFactory
{
    public bool IsReusable => false;

    public IFilterMetadata CreateInstance(IServiceProvider
serviceProvider) =>
        new InternalResponseHeaderFilter();

    private class InternalResponseHeaderFilter : IActionFilter
    {
        public void OnActionExecuting(ActionExecutingContext context) =>
            context.HttpContext.Response.Headers.Add(
                nameof(OnActionExecuting), name-
of(InternalResponseHeaderFilter));

        public void OnActionExecuted(ActionExecutedContext context) { }
    }
```

O filtro é aplicado no seguinte código:

C#

```
[ResponseHeaderFilterFactory]
public IActionResult Index() =>
    Content($"- {nameof(FilterFactoryController)}.{nameof(Index)}");
```

## IFilterFactory implementado em um atributo

Filtros que implementam `IFilterFactory` são úteis para filtros que:

- Não exigem a passagem de parâmetros.
- Tenha dependências de construtor que precisem ser atendidas pela DI.

TypeFilterAttribute implementa IFilterFactory. `IFilterFactory` expõe o método

CreateInstance para criar uma instância de IFilterMetadata. `CreateInstance` carrega o tipo especificado do contêiner de serviços (DI).

C#

```
public class SampleActionTypeFilterAttribute : TypeFilterAttribute
{
    public SampleActionTypeFilterAttribute()
         : base(typeof(InternalSampleActionFilter)) { }

    private class InternalSampleActionFilter : IActionFilter
    {
        private readonly ILogger<InternalSampleActionFilter> _logger;

        public
InternalSampleActionFilter(ILogger<InternalSampleActionFilter> logger) =>
            _logger = logger;

        public void OnActionExecuting(ActionExecutingContext context)
        {
            _logger.LogInformation(
                $"- {nameof(InternalSampleActionFilter)}.{name-
of(OnActionExecuting)}");
        }

        public void OnActionExecuted(ActionExecutedContext context)
        {
            _logger.LogInformation(
                $"- {nameof(InternalSampleActionFilter)}.{name-
of(OnActionExecuted)}");
        }
    }
}
```

O código a seguir mostra três abordagens para aplicar o filtro:

C#

```
[SampleActionTypeFilter]
public IActionResult WithDirectAttribute() =>
    Content($"- {nameof(FilterFactoryController)}.{name-
of(WithDirectAttribute)}");

[TypeFilter(typeof(SampleActionTypeFilterAttribute))]
public IActionResult WithTypeFilterAttribute() =>
    Content($"- {nameof(FilterFactoryController)}.{name-
of(WithTypeFilterAttribute)}");

[ServiceFilter(typeof(SampleActionTypeFilterAttribute))]
public IActionResult WithServiceFilterAttribute() =>
    Content($"- {nameof(FilterFactoryController)}.{name-
```

```
of(WithServiceFilterAttribute)}");
```

No código anterior, a primeira abordagem para aplicar o filtro é preferencial.

# Usar middleware no pipeline de filtro

Filtros de recursos funcionam como middleware, no sentido em que envolvem a execução de tudo o que vem depois no pipeline. Mas os filtros diferem do middleware, pois fazem parte do runtime, o que significa que eles têm acesso ao contexto e aos constructos.

Para usar o middleware como um filtro, crie um tipo com um método `Configure` que especifica o middleware para injeção no pipeline de filtros. O exemplo a seguir usa middleware para definir um cabeçalho de resposta:

C#

```
public class FilterMiddlewarePipeline
{
    public void Configure(IApplicationBuilder app)
    {
        app.Use(async (context, next) =>
        {
            context.Response.Headers.Add("Pipeline", "Middleware");

            await next();
        });
    }
}
```

Use MiddlewareFilterAttribute para executar o middleware:

C#

```
[MiddlewareFilter(typeof(FilterMiddlewarePipeline))]
public class FilterMiddlewareController : Controller
{
    public IActionResult Index() =>
        Content($"- {nameof(FilterMiddlewareController)}.{name-
of(Index)}");
}
```

Filtros de middleware são executados no mesmo estágio do pipeline de filtros que filtros de recurso, antes do model binding e depois do restante do pipeline.

# Acesso thread-safe

Ao passar uma *instância* de um filtro para `Add`, em vez de seu `Type`, o filtro é um singleton e **não** é thread-safe.

# Recursos adicionais

- Exibir ou baixar exemplo (como baixar).
- Filtrar métodos para Razor Páginas no ASP.NET Core